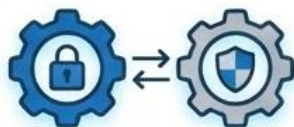


Introducción a ASP.NET Core Identity

Autenticación y autorización en aplicaciones .NET modernas



Objetivos de la Sesión



AuthN

AuthZ

1. Comprender autenticación vs autorización

- Diferenciar quién eres de qué puedes hacer.



2. Conocer qué es ASP.NET Core Identity

- Explorar el sistema de identidad integrado de Microsoft.



3. Comparar Identity con Auth0 y otras soluciones

- Evaluar cuándo usar Identity vs proveedores externos.



4. Entender escenarios reales (Web, API, Mobile)

- Aplicar conceptos a diferentes arquitecturas de aplicación.

El Problema que Resolvemos



1. Registro y login de usuarios

- Implementación segura de formularios de alta y acceso.
- Gestión de credenciales y recuperación de contraseñas.



2. Control de acceso seguro

- Autenticación confiable de la identidad del usuario.
- Prevención de accesos no autorizados a recursos.



3. Manejo de sesiones y tokens

- Mantenimiento seguro del estado del usuario.
- Generación y validación de tokens (JWT, cookies).



4. Seguridad en múltiples aplicaciones

- Autenticación centralizada para diversos clientes.
- Federación de identidad y SSO.

Autenticación vs Autorización



Autenticación (AuthN)

- Verificación de la identidad del usuario. **¿Quién eres?**



Identificado



Autorización (AuthZ)

- Determinación de permisos y privilegios. **¿Qué puedes hacer?**



Identificado



Recursos



Gatekeeper



Acceso
Permitido
Acceso
Denegado

Ejemplo: Usuario Autenticado SIN Permisos

Logged In



Datos Confidenciales

Error 403: Forbidden (Acceso Denegado).
Identidad confirmada, pero sin privilegios suficientes.

Modelos de Autenticación



Cookies (Web tradicional)



- Almacenada en el navegador del cliente.
- El servidor mantiene el estado de la sesión.
- Enviada automáticamente en cada petición (misma origen).
- Vulnerable a ataques CSRF si no se protege.



JWT Tokens (APIs)



- Almacenado por el cliente (localStorage, cookie).
- Autocontenido, no requiere estado en el servidor.
- Debe ser enviado explícitamente (Header Authorization).
- Vulnerable a XSS si se almacena en localStorage.

Diferencias: Sesión vs. Token



Sesión (Stateful)

- Estado en servidor
- Escalabilidad limitada
- Revocación inmediata



Token (Stateless)

- Sin estado en servidor
- Fácilmente escalable
- Difícil de revocar antes de expirar

¿Qué es ASP.NET Core Identity?



Framework de Gestión de Usuarios

- Biblioteca integral de .NET para manejar toda la lógica de usuarios, autenticación y autorización.

Funcionalidades Clave



- **Registro:** Creación de nuevas cuentas.



- **Login:** Inicio de sesión seguro.



- **Roles:** Agrupación de permisos (ej. Admin, Usuario).



- **Claims:** Información del usuario (ej. Email, Edad).



Funcionalidades

- Biblioteca de ASP.NET para (ej. adminzante).

Integración Nativa

- Diseñado para funcionar sin problemas dentro del ecosistema de ASP.NET Core, facilitando la configuración y el uso.

Componentes Clave de Identity



UserManager

- Gestión de usuarios (crear, leer, actualizar, eliminar). Maneja claims, passwords, etc.



RoleManager

- Gestión de roles (crear, asignar, eliminar). Define permisos y grupos.



SignInManager

- Maneja el inicio y cierre de sesión (autenticación).
- Valida credenciales, genera cookies.



Password hashing y políticas



Password hashing

- Seguridad de contraseñas. Algoritmos de hash robustos, reglas de complejidad.



EF Core Stores

- Capa de acceso a datos. Implementación de almacenamiento usando Entity Framework Core.

Flujo Básico con Identity



Seguridad y Buenas Prácticas



Políticas de contraseña

- Complejidad (longitud, caracteres).
- Rotación y prevención de reutilización.



Lockout

- Bloqueo temporal tras intentos fallidos.
- Prevención de ataques de fuerza bruta.



Confirmación de email

- Verificación de identidad y buzón.
- Requisito para activación de cuenta.



MFA / 2FA

- Autenticación multifactor obligatoria.
- Uso de SMS, aplicaciones o tokens.



Auditoría

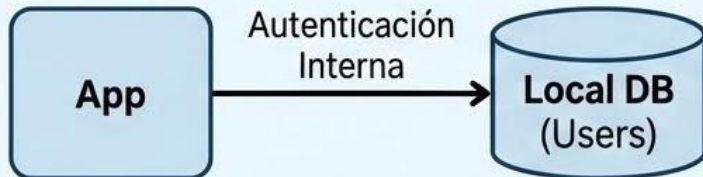
- Registro de eventos de seguridad.
- Monitoreo y análisis forense.

Identity NO es OAuth2 / OpenID Connect



Identity: Gestión de Usuarios Locales

- Base de datos de usuarios propia de la aplicación.
- Credenciales (username/password) almacenadas localmente (hasheadas).
- Control total sobre el ciclo de vida del usuario.



VS



OAuth2 / OpenID Connect: Autenticación Federada

- Delega la autenticación en un Proveedor de Identidad (IdP) externo.
- La aplicación NO maneja credenciales; recibe un token.
- Protocolos estándar para autorización y autenticación.



Panorama de Soluciones de Identidad



ASP.NET Core Identity

- Gestión local de usuarios (cuentas, contraseñas, roles).
- Integrado directamente en aplicaciones ASP.NET Core.
- Ideal para aplicaciones monolíticas o autocontenidas.



Auth0

- Identity as a Service (IDaaS) líder en el mercado.
- Universal Login para múltiples aplicaciones.
- Soporte extensivo para conexiones sociales y empresariales.



Microsoft Entra ID

- Solución de identidad y acceso basada en la nube de Microsoft.
- Anteriormente conocido como Azure Active Directory (Azure AD).
- Fuerte integración con el ecosistema Microsoft 365 y Azure.



Keycloak

- Gestión de identidad y acceso (IAM) de código abierto.
- Single Sign-On (SSO), corretaje de identidad y login social.
- Opciones de despliegue self-hosted o gestionado.



IdentityServer

- Framework open-source para construir IdPs personalizados.
- Implementa OpenID Connect y OAuth 2.0.
- Gran flexibilidad para escenarios complejos y federación.

Identity vs Auth0

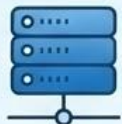


ASP.NET Core Identity (Autogestionado)



Control y Gestión

- Control total sobre el código y ciclo de vida.
- Requiere desarrollo y mantenimiento continuo.
- Alta capacidad de personalización.



Despliegue e Infraestructura

- Autogestionado (On-Premise o VM).
- Administración de parches y seguridad por el equipo.
- Base de datos de usuarios propia.



Costos y Dependencia

- Costo en horas de desarrollo y operación.
- Baja dependencia externa (código propio).
- Sin costos de licencia por usuario.

VS



Auth0 (IDaaS - SaaS)



Rapidez y Abstracción

- Implementación muy rápida (SDKs listos).
- Abstrae la complejidad de la seguridad.
- Configuración sobre desarrollo.



Despliegue e Infraestructura

- Software as a Service (SaaS).
- Infraestructura gestionada por el proveedor.
- Escalabilidad y alta disponibilidad automáticas.



Costos y Dependencia

- Costo recurrente por suscripción (tiers).
- Alta dependencia del proveedor externo.
- Riesgo de "vendor lock-in".

Terminología Clave



Claims

- Declaraciones sobre un usuario (nombre, email, edad).
- Pares clave-valor dentro de un token.
- La base para la toma de decisiones de autorización.



Roles

- Conjunto de permisos o funciones asignados a un usuario.
- Ejemplos: 'Admin', 'Usuario', 'Gestor'.
- Simplifica la gestión de permisos en grupo.



Policies

- Reglas de autorización que evalúan claims o roles.
- Definen quién puede hacer qué y bajo qué condiciones.
- Permiten una lógica de acceso más granular y flexible.



Scopes

- Permisos que una aplicación solicita (ej. "read:profile").
- Definen el nivel de acceso a los recursos.
- El usuario consiente estos alcances durante el login.



Audience

- El destinatario previsto del token (resource server).
- El servicio que consumirá y validará el token.
- El token debe ser rechazado si la audiencia no coincide.

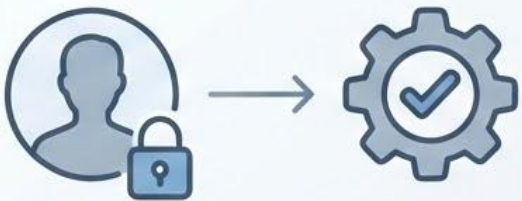


Issuer

- La entidad que emitió y firmó digitalmente el token.
- El servidor de autorización en el que se confía (ej. Auth0, IdentityServer).
- Su identidad se verifica mediante su clave pública.

Conclusión

Identity = Control



- Gestión centralizada de accesos y permisos.
- Mayor seguridad y cumplimiento normativo.
- Control granular sobre quién accede a qué.

Federación y SSO (Auth0, Entra, Keycloak)



- Inicio de sesión único (SSO) para múltiples aplicaciones.
- Federación de identidades entre organizaciones.
- Simplifica la experiencia del usuario y reduce la gestión de contraseñas.